Data Extraction Methods Overview

Explore various data extraction methods critical for ETL processes. Learn to use SQL for querying databases, Python for web scraping and API access, and libraries like pandas for extracting data from files. Understand how to connect to databases with Python drivers for effective data retrieval.

There are several methods for extracting data, depending on the type of data, the source of the data, and the purpose of the extraction. This lesson intends to provide a brief overview of some of the most popular ones.

SQL queries

The most common method of extracting data is via SQL. SQL queries extract structured data from relational databases and data warehouses. SQL is a powerful tool that can be used to solve various tasks, from simple data retrieval to complex data analysis and management.

Basic SQL syntax is easy to learn, and structured data is easy to process, making SQL a very common method for extracting data. After querying the required data from the database, we can use SQL to export the query as a CSV file.

- We use a `select` statement to choose the relevant columns to retrieve data from a database.
- We then add `from` to choose the table.
- To filter the retrieved data, we use the `where` clause.
- We group rows using `group by` and then aggregate them using functions like `count`, `min`, `max`, `sum`, `average`, and more.

Ask

- Finally, we use a `copy` statement to export tables or query results.

1.Se...

```
create table customers(
    customer_id int not null primary key,
    first_name varchar(30) not null,
    last_name varchar(30) not null,
    email varchar(45) not null,
    country varchar(30) not null
);

insert into customers
values
(1, 'Ryan', 'Dawson', 'ryan@dawson.com', 'USA'),
(2, 'Noah', 'Richardson', 'noah@richardson.com', 'USA'),
(3, 'Mathias', 'Boudet', 'mathias@boudet.com', 'France')
```

Run

SQL basics—select

Programming language

A programming language like Python is another powerful tool for extracting data. It can be a great choice as it can easily connect to applications or databases and pull data. Furthermore, the same programming language can also be used to process and transform the data. Check out the following examples of extracting data using Python.

Extracting data using web scraping/API

Python can send HTTP requests to web pages and REST API services, process the response, and extract data. For example, we can extract the HTML content of regular web pages on the internet, such as this:

Ask

Web page to scrape

In this script, we send an HTTP request to the official Python website. We then parse the returned HTML content and extract the contents under the title “The Zen of Python.”

main.py Python 3.10.4

```
import requests
from bs4 import BeautifulSoup

url = "https://peps.python.org/pep-0020/"

# Send a request to the URL and get the response
response = requests.get(url)

# Parse the HTML content of the page
soup = BeautifulSoup(response.content, "html.parser")
```

Ask

```
# Find the element containing the text you want to extract
```

Run

Extracting via web scraping

- **Line 7:** We send a `get` request and receive a response.
- **Line 10:** We parse the response using the HTML parser from `BeautifulSoup`.
- **Line 13:** We scan the response and retrieve elements of the type `div` and the class `highlight`.

Extracting data from files

Another form of data extraction is handling and extracting data from files. For example, we can use the Python library pandas to easily extract data from various files, such as CSV, JSON, XML, and Excel files, into a pandas DataFrame.

A pandas DataFrame is a powerful two dimensional tabular data structure commonly used for data analysis and machine learning applications. For example, the following script extracts the CSV file we exported earlier into a pandas DataFrame.

It uses the `pd.read_csv()` method on **line 4** to read a CSV file into a pandas DataFrame.

main.py Python 3.10.4

```
import pandas as pd

# Extract the content of query_results.csv into a pandas DataFrame
df = pd.read_csv('tmp/query_results.csv')

# Print the DataFrame
print(df)
```

Ask

Run

Extracting from a CSV file

After loading the data into the pandas DataFrame, we can use a suite of methods to access different parts of the DataFrame and process the data. Let's see some examples:

main.py Python 3.10.4

```
# Printing a status report of the DataFrame
status_report = df.describe()
print(f"DataFrame status report:\n{status_report}\n")

# Printing the column: Number of Customers
number_of_customers = df.loc[:,"Number of Customers"].values
print(f"Number of customers columns: {number_of_customers}\n")

# Printing the total number of customers
sum_of_customers = df.loc[:,"Number of Customers"].sum()
print(f"Total customers: {sum_of_customers}\n")

# Creating a DataFrame of customers from France
```

Run

- **Line 2:** We generate a status report about the numerical columns in the DataFrame that includes statistics such as the number of non-null values, mean, standard deviations, and more.
- **Line 6:** We extract the column `Number of Customers` from the DataFrame.
- **Line 10:** We calculate the sum of the column `Number of Customers`.
- **Line 14:** We generate a new DataFrame of customers only from Franc
- **Line 18**: We generate a new DataFrame of customers only from France or the USA.

Ask

- **Line 22:** We calculate the average number of customers from France and Japan.

Extracting data from databases

A programming language like Python can also be used to form a connection to a database and interact with it. Several popular database drivers and libraries can facilitate the connection between our local machine and a database server. Here are some examples:

- `psycopg2`: A PostgreSQL driver that allows us to connect to and query a PostgreSQL database using Python
- `pymysql`: A MySQL driver that allows us to connect to and query a MySQL database using Python
- `pyodbc`: A generic database driver that allows us to connect to and query various databases using the ODBC protocol
- `pymongo`: A MongoDB driver that allows us to connect to and query a MongoDB database using Python

For example, the following script uses `psycopg2` to connect to the PostgreSQL server from the previous session and query the database. The script will select all customers where `country` is equal to `France`.

main.py Python 3.10.4

```
import psycopg2

# Connect to the database
conn = psycopg2.connect(host='localhost', port = 5432, database='postgres

# Create a cursor
cur = conn.cursor()

# Execute a SQL query
cur.execute("SELECT * FROM customers where country = 'France'; ")
```

Ask

```
# Fetch the results of the query
```

Run

Extracting data from PostgreSQL

- **Line 4:** We form a connection to an active PostgreSQL database.
- **Line 7:** We create a cursor object, which is used to execute actions on the database.
- **Line 10:** We execute a query to select all customers from France.
- **Line 13**: We fetch query results.
- **Lines 16–17**: We iterate over the query results and print each row.

Other methods for extracting data include:

- Extracting from NoSQL databases using specialized SQL syntax (e.g., CQL for Apache Cassandra, MQL for MongoDB)
- Using a programming language to extract streaming data from Apache Kafka or other message brokers
- Using a programming language, the command line, or a graphical user interface to extract data from proprietary cloud solutions (e.g., GCP, Azure, AWS)

Ask